§4.1 Motivation und Konzept

Problem

- Bei rekursiven Funktionen kann es zu mehrfacher Berechnung des gleichen Funktionswerts kommen.

Beispiel

- Die Fibonacci-Zahlen sind definiert als:

$$F_0 = 0, F_1 = 1, \quad F_i = F_{i-1} + F_{i-2}, \quad i \geq 2.$$

```
int fib(int n)
{
   if (n==0) return 0;
   if (n==1) return 1;
   return fib(n-1) + fib(n-2);
}
```

Beispiel

- Fibonacci-Zahlen

```
int fib[N+1];

fib[0] = 0;
fib[1] = 1;

for (int n=2; n<=N; n++)
   fib[n] = fib[n-1]+fib[n-2];
```

Tabelle

Basisfälle

Rekursionsschritt aufwärts angewendet

Analyse

- Dynamisch programmiert: $O(n)$
- Rekursiv programmiert: $O(1.6^n)$
  - Master Theorem für subtract-and-conquer Algorithmen: $O(2^n)$

§4.2 Binomialkoeffizienten

Problem

- Eine Teilmenge („Ziehen ohne Zurücklegen“) von k verschiedenen Elementen aus einer Menge von n Elementen heißt **Kombination**.
- Die Anzahl der verschiedenen Kombinationen von k aus n Elementen ist:

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!}, n \geq k \geq 0.$$

- Beispiel: Die Anzahl der möglichen Lottokombinationen ist $\binom{49}{6}$.

§4.2 Binomialkoeffizienten

Algorithmus

```
int komb[n+1][n+1];

// Basisfall (1):
for (int i = 0; i <= n; i++)
   komb[i][0] = 1;

// Basisfall (2):
for (int i = 0; i <= n; i++)
   komb[i][i] = 1;

// Rekursionsschritt:
for (int i = 2; i <= n; i++)
   for (int k = 1; k < i; k++)
      komb[i][k] = komb[i-1][k] + komb[i-1][k-1];
```

Tabelle zum Speichern der Zwischenergebnisse:

$$\texttt{komb[n][k]} = \binom{n}{k}$$

§4.2 Binomialkoeffizienten

Beispiel

- Für **`N=4`** ergibt die folgende Tabelle **`komb`**:

n \ k	0	1	2	3	4
0	1				
1	1	1			
2	1	2	1		
3	1	3	3	1	
4	1	4	6	4	1

Basisfälle

Rekursion

```
komb[n][k] = komb[n-1][k  ]+
             komb[n-1][k-1];
```

- Bemerkung: In der Tabelle steht am Ende das Pascalsche Dreieck.
- Laufzeit: $O(n^2)$
- Speicher: $O(n^2)$

§4.3 Alle kürzesten Wege in Graphen

- Man kommt jedoch auch mit genau einer Matrix aus!

  Es gilt nämlich

$$D^k(i,j) = \min\{D^{k-1}(i,j),\ D^{k-1}(i,k)+D^{k-1}(k,j)\}$$
$$= \min\{D^{k-1}(i,j),\ D^{k}\ (i,k)+D^{k}\ (k,j)\}, \quad (3)$$

  denn $D^k(i,k)=D^{k-1}(i,k)$ und $D^k(k,j)=D^{k-1}(k,j)$, d.h.

  - Wege mit v_k als Anfangs- oder Endpunkt bleiben in der k-ten Iteration unverändert.

- Damit kann (3) durch folgende Programmanweisung berechnet werden:

```
D[i][j] = min(D[i][j], D[i][k] + D[k][j]).
```

Alter Wert von `D[i][j]` ist $D^{k-1}(i,j)$.

Neuer Wert von `D[i][j]` ist $D^k(i,j)$.

§4.3 Alle kürzesten Wege in Graphen

- Damit lässt sich **`D[i][j]`** wie folgt berechnen:

```
for (int i=1; i<=n; i++) {  // Berechne D^0
   D[i][i]=0;
   for (int j=1; j<=n; j++)
      if ( i!=j ) D[i][j] = c(v_i,v_j);
}
for (int k=1; k<=n; k++)    // Berechne D^k
   for (int i=1; i<=n; i++)
      for (int j=1; j<=n; j++)
         if ( D[i][j] > D[i][k]+D[k][j])
              D[i][j] = D[i][k]+D[k][j];
```

```
D[i][j] = min(D[i][j] ,D[i][k]+D[k][j]);
```

§4.3 Alle kürzesten Wege in Graphen

- Zur Speicherung der gefundenen Wege wird eine Vorgängermatrix **P** eingesetzt:

  P[i][j] = Vorgängerknoten von $\mathbf{v_j}$ im kürzesten Weg von $\mathbf{v_i}$ nach $\mathbf{v_j}$.

- Damit kann der kürzeste Weg von **s** nach **z** ausgegeben werden (in umgekehrter Reihenfolge):

```
v = z; cout << v << endl;
while ( v!=s ) {
   v = P[s][v]; cout << v << endl;
}
```

§4.3 Alle kürzesten Wege in Graphen

```
void allShortestPaths( DiGraph G, int D[][], Vertex P[][])
{
   for (int i=1; i<=n; i++) {      // Initialisiere D und P
      D[i][i]=0;
      for (int j=1; j<=n; j++) {
         if (       i!=j )  D[i][j] = c(v_i,v_j);
         if (D[i][j]!=∞ )  P[i][j] = i; else P[i][j] = 0;
      }
   }
   for (int k=1; k<=n; k++)        // Berechne D^k
      for (int i=1; i<=n; i++)
         for (int j=1; j<=n; j++)
             if ( D[i][j] > D[i][k]+D[k][j]) {
                  D[i][j] = D[i][k]+D[k][j];
                  P[i][j] = P[k][j];
             }
}
```

Analyse

- Enthält der Graph Zyklen mit negativer Länge, dann wird ein Diagonalelement von **D** negativ.
  - Der Algorithmus kann leicht um eine passende Überprüfung mit entsprechender Ausgabe erweitert werden.
- Aufwand: Die dreifach geschachtelten **for**-Schleifen ergeben:

$$O(|V|^3).$$

Bemerkung

- Dieser Algorithmus geht auf Robert Floyd und Stephen Warshall (1962) zurück.

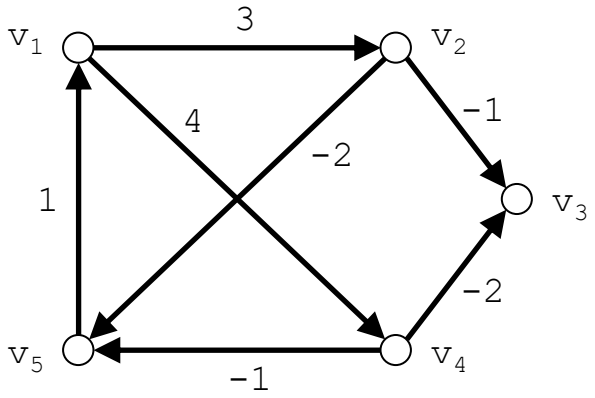

R.W. Floyd



S. Warshall

§4.3 Alle kürzesten Wege in Graphen

Beispiel ([Turau])



D^0

0	3	∞	4	∞
∞	0	-1	∞	-2
∞	∞	0	∞	∞
∞	∞	-2	0	-1
1	∞	∞	∞	0

D^1

0	3	∞	4	∞
∞	0	-1	∞	-2
∞	∞	0	∞	∞
∞	∞	-2	0	-1
1	4	∞	5	0

D^2

0	3	2	4	1
∞	0	-1	∞	-2
∞	∞	0	∞	∞
∞	∞	-2	0	-1
1	4	3	5	0

D^3

0	3	2	4	1
∞	0	-1	∞	-2
∞	∞	0	∞	∞
∞	∞	-2	0	-1
1	4	3	5	0

D^4

0	3	2	4	1
∞	0	-1	∞	-2
∞	∞	0	∞	∞
∞	∞	-2	0	-1
1	4	3	5	0

D^5

0	3	2	4	1
-1	0	-1	3	-2
∞	∞	0	∞	∞
0	3	-2	0	-1
1	4	3	5	0

P^0

0	1	0	1	0
0	0	2	0	2
0	0	0	0	0
0	0	4	0	4
5	0	0	0	0

P^1

0	1	0	1	0
0	0	2	0	2
0	0	0	0	0
0	0	4	0	4
5	1	0	1	0

P^2

0	1	2	1	2
0	0	2	0	2
0	0	0	0	0
0	0	4	0	4
5	1	2	1	0

P^3

0	1	2	1	2
0	0	2	0	2
0	0	0	0	0
0	0	4	0	4
5	1	2	1	0

P^4

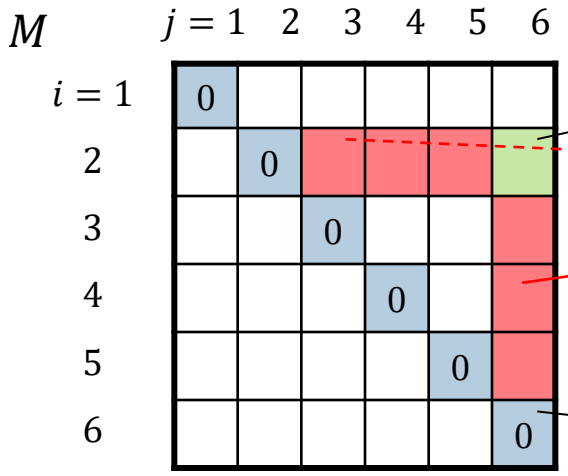
0	1	2	1	2
0	0	2	0	2
0	0	0	0	0
0	0	4	0	4
5	1	2	1	0

P^5

0	1	2	1	2
5	0	2	1	2
0	0	0	0	0
5	1	4	0	4
5	1	2	1	0

Beispiel

- Bemerkung: Die Rekursion geht über die Länge der Matrizenkette $m = j - i$.



Rekursion: $M(i,j)$ berechnet sich aus $M(i,k)$ und $M(k+1,j)$ mit $i \leq k < j$.

Basisfälle: $M(i,j) = 0$, für $i = j$.

§4.4 Kettenmultiplikation von Matrizen

```
// Tabellen:
int M[n+1][n+1], OptKl[n+1][n+1];

// Basisfälle m = j - i = 0:
for (int i = 1; i <= n; i++) M[i][i]=0;

// Rekursion m = j - i > 0:
for (int m = 1; m < n; m++) {          // Länge der Matrixkette
   for (int i = 1; i <= n-m; i++) {    // Position der ersten    Kl.
      int j = i + m;                   // Position der letzten   Kl.
      M[i][j] = ∞;
      for (int k = i; k < j; k++) {    // Position der mittleren Kl.
         int W = M[i][k] + M[k+1][j] + r_{i-1}·r_k·r_j;
         if (W < M[i][j]) { M[i][j] = W; OptKl[i][j] = k; }
      }
   }
}
```

Im Feld `OptKl` wird die optimale Klammerung gespeichert.

`OptKl[i][j]=k` gibt an, dass die Klammerung nach Matrix A_k erfolgt:

$$A_i \cdot \ldots \cdot A_j = (A_i \cdot \ldots \cdot A_k) \cdot (A_{k+1} \cdot \ldots \cdot A_j)$$

§4.4 Kettenmultiplikation von Matrizen

Analyse

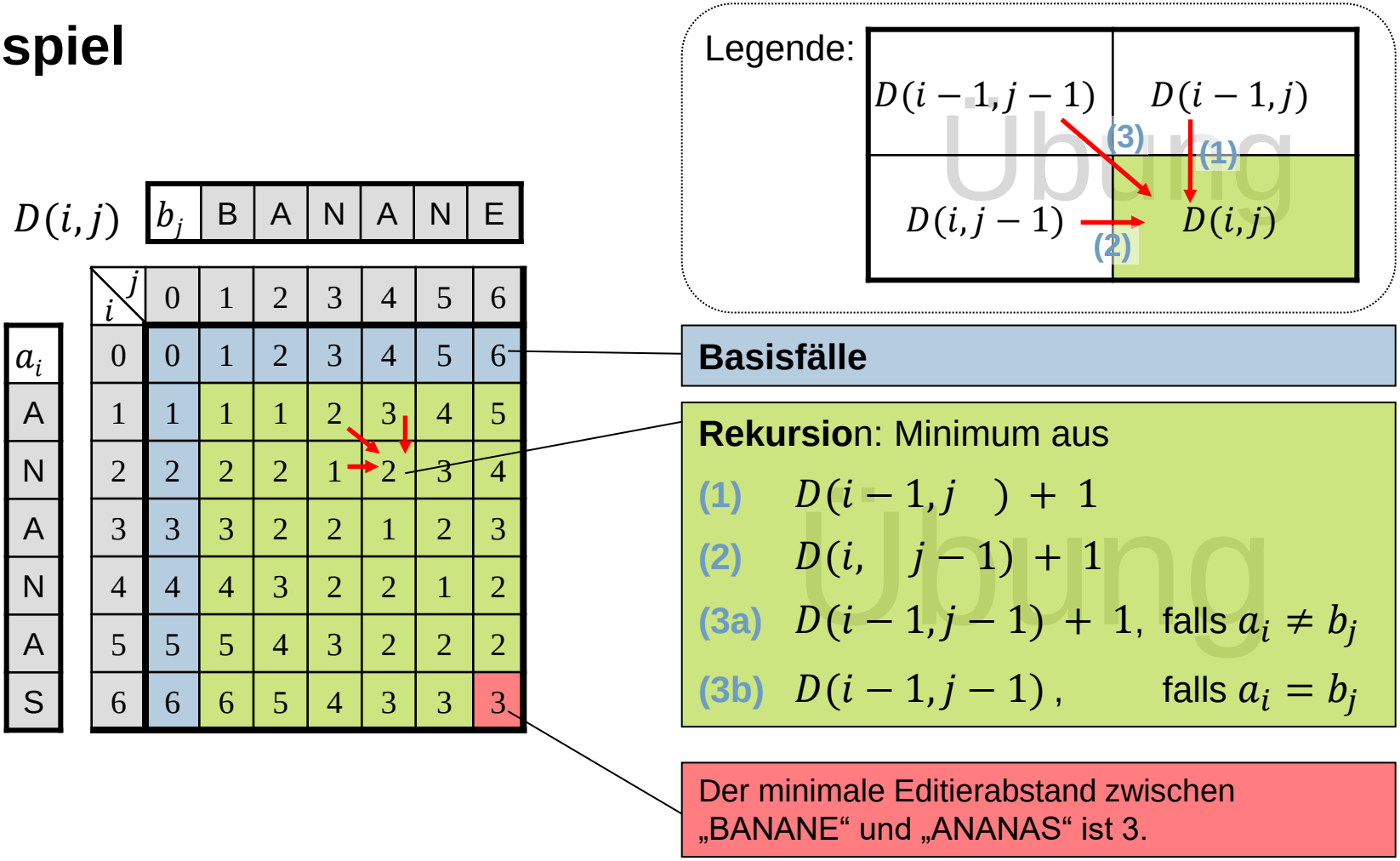
- Die äußeren beiden Schleifen werden $O(n^2)$ mal durchlaufen.
- Die innerste Schleife wird m-mal durchlaufen.
- Gesamte Laufzeit: $O(n^3)$.
- Gesamter Speicherverbrauch: $O(n^2)$.

Bemerkung

- Verbesserung: Yao [1982], $O(n^2)$
  - Dynamisches Programm basierend auf geometrischer Interpretation.
- Verbesserung: Hu/Shing [1982/84], $O(n \log n)$
  - Geometrischer Algorithmus zur Zerlegung von Polygonen.

§4.5 Editierabstand

Beispiel



Algorithmus zum Aufbau der Tabelle

```
// Tabelle:
int D[n+1,m+1];

// Basisfälle:
for (int i=0; i<=n; i++) D[i,0]=i;
for (int j=0; j<=m; j++) D[0,j]=j;

// Rekursion i,j > 0:
for (int i=1; i<=n; i++) {
   for (int j=1; j<=m; j++) {
      int t  = (a[i]==b[j])? 0: 1;
      D[i,j] = min(D[i,j-1],D[i-1,j  ]  ) + 1;
      D[i,j] = min(D[i,j  ],D[i-1,j-1]+t);
   }
}
```

Übung

§4.5 Editierabstand

- Wie lassen sich aus der Tabelle die Editieroperationen ablesen?
  - Speichere für jedes $D(i,j)$ ab, wie sich das Minimum ergeben hat:
    - **(1)** Minimum ist $D(i-1,j)+1$:
    - **(2)** Minimum ist $D(i,j-1)+1$:
    - **(3a)** Es ist $a_i \neq b_j$ und das Minimum ist $D(i-1,j-1)+1$:
    - **(3b)** Es ist $a_i = b_j$ und das Minimum ist $D(i-1,j-1)$:
  - Basisfall-Elemente können wie **(1)** bzw. **(2)** behandelt werden.



$D(i,j)$

b_j	B	A	N	A	N	E

i \ j	0	1	2	3	4	5	6
a_i	0	1	2	3	4	5	6
A	1	1	1	2	3	4	5
N	2	2	2	2	1	2	3
A	3	3	3	2	2	1	2
N	4	4	4	3	2	2	1
A	5	5	5	4	3	2	2
S	6	6	6	5	4	3	3

Übung

Algorithmus zum Finden der Editieroperationen

```
string path;
int    i=n, j=m;
while (i>0 && j>0) {
   int t = (a[i]==b[j])? 0: 1;
   if (D[i,j]==D[i-1,j-1]+t) {
      if (t) path.append(↘); else path.append(→);
      i--; j--;
   } else if (D[i,j]==D[i-1,j]+1) {
      path.append(↓); i--;
   } else {
      path.append(→); j--;
   }
}
while (i>0) { path.append(↓); i--; }
while (j>0) { path.append(→); j--; }
```

§4.5 Editierabstand

Analyse

- Aufbau der Tabelle:
  - Basisfälle: $O(n + m)$.
  - Rekursion: $O(n \cdot m)$.
  - Gesamt: $O(n \cdot m)$.
  - Speicherbedarf: $O(n \cdot m)$.

Übung

- Berechnung der Editieroperationen:
  - Finden der Editieroperationen: $O(n + m)$.
  - Gesamt: $O(n \cdot m)$.

Bemerkung:

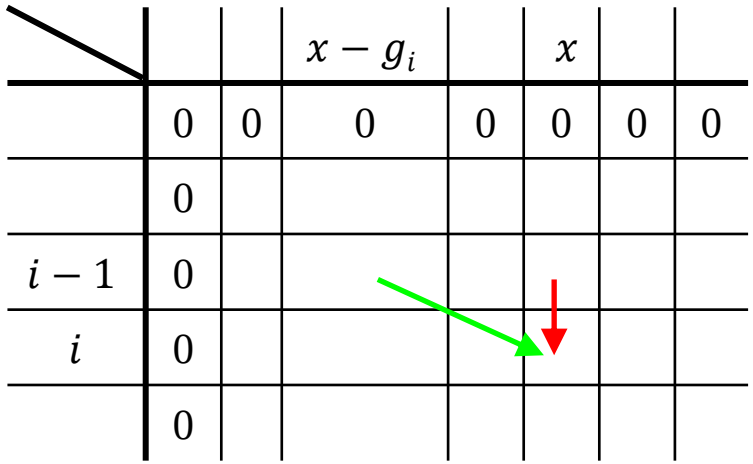
- Dieser Algorithmus geht auf Needleman/Wunsch [1970] zurück.

Das 0-1-Rucksackproblem

- Algorithmus

```
int P[n+1,G+1];
for (int x=0; x<=G; x++) P[0,x] = 0; // Basisfall für i=0
for (int i=0; i<=n; i++) P[i,0] = 0; // Basisfall für x=0
// Rekursion für i,x>0
for (int i=1; i<=n; i++) {
   for (int x=1; x<=G; x++) {
      P[i,x] = P[i-1,x];
      if ( x-g_i>=0 ) {
         double t = w_i + P[i-1,x-g_i];
         P[i,x]   = max(P[i,x],t);
      }
   }
}
```

Schematisch:



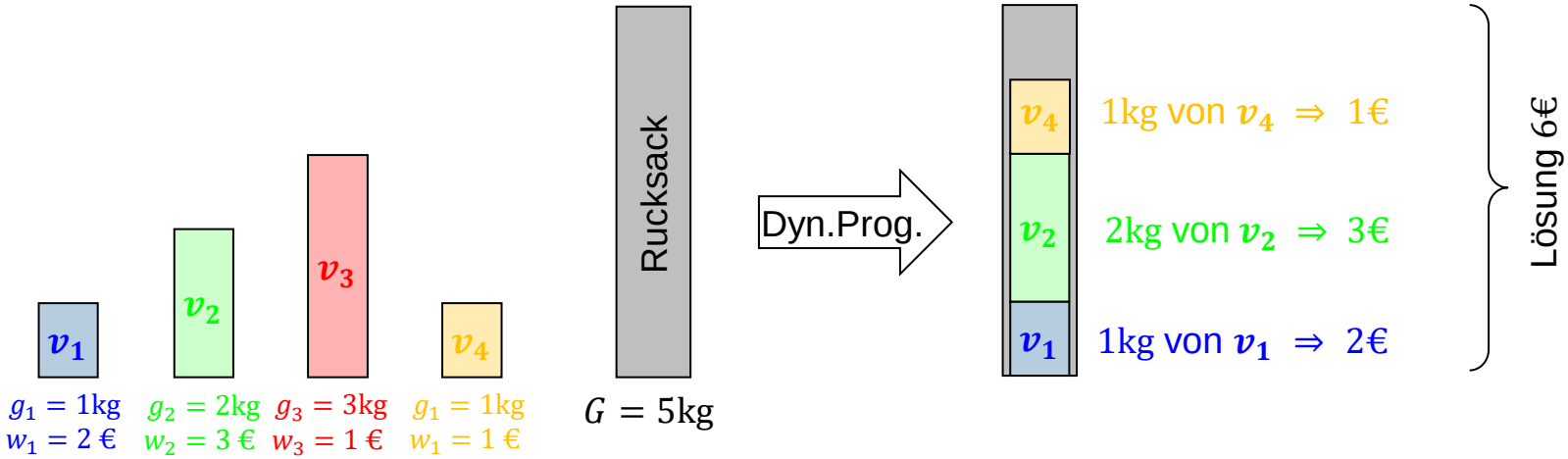
Das 0-1-Rucksackproblem

- Beispiel

i \ x	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	2	2	2	2	2
2	0	2	3	5	5	5
3	0	2	3	5	5	5
4	0					

i \ x	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	2	2	2	2	2
2	0	2	3	5	5	5
3	0	2	3	5	5	5
4	0	2	3	5	6	6

Füllung aus Profitmatrix rekonstruieren



Analyse

- Der Greedy-Algorithmus für das fraktionale Rucksackproblem hat wegen des Sortierens einen Aufwand von

$$T(n) = O(n) + O(n \log n) = O(n \log n).$$

- Das dynamische Programm für das 0-1-Rucksackproblem hat einen Aufwand von

$$T(n) = O(n) + O(G) + O(n \cdot G) = O(n \cdot G).$$

Widerspruch zur NP-Eigenschaft des Rucksackproblems?

- Der Algorithmus zur Lösung des 0-1-Rucksackproblems ist ein numerischer Algorithmus, d.h. er ist polynomial im numerischen Wert der Eingabe.

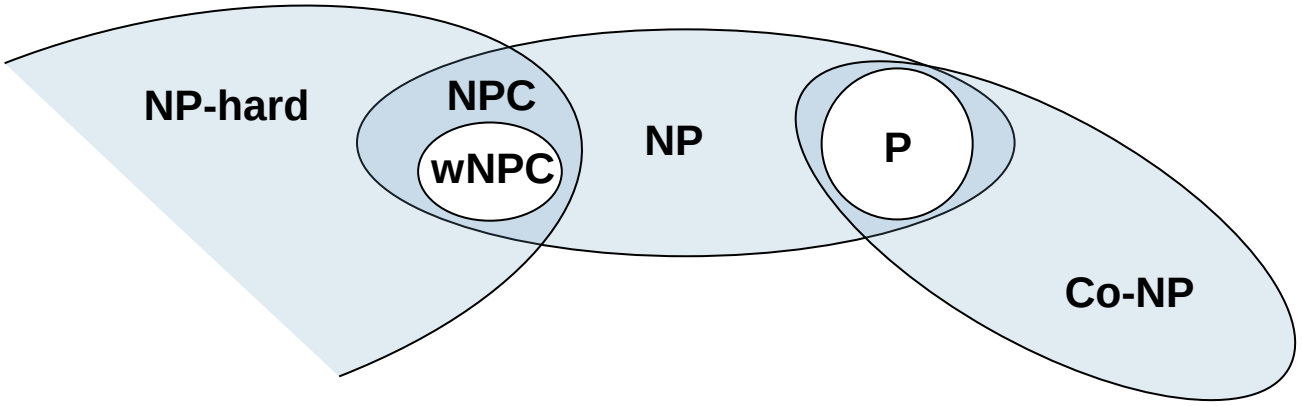
	Laufzeit im Wert n, m	**Laufzeit in Anzahl der Stellen a, b**	**Zuwachs bei $a \rightarrow a+1$**
Addition $n + m$	$m + n$ linear $\Rightarrow O(1)$	$m = 10^a, n = 10^b$ $\Rightarrow O(\max(a, b))$	$+1$
Multiplikation $n \cdot m$	$m \cdot n$ $\Rightarrow O(1)$	$m = 10^a, n = 10^b$ $\Rightarrow O(a \cdot b)$	$+b$
Primzahltest von n	teile durch $2, \ldots, \sqrt{n}$ $\Rightarrow O(\sqrt{n})$	$n = 10^a$ $\Rightarrow 10^{a/2} = O(10^a)$	$\cdot 10$
0-1-Rucksack	n Objekte, G Größe $\Rightarrow O(n \cdot G)$	$n, G = 10^a$ $\Rightarrow O(n \cdot 10^a)$	$\cdot 10$

Widerspruch zur NP-Eigenschaft des Rucksackproblems?

- Der Algorithmus zur Lösung des 0-1-Rucksackproblems ist ein numerischer Algorithmus, d.h. er ist polynomial im numerischen Wert der Eingabe.
- **Aber:** Er ist exponentiell in der Anzahl der Stellen der Eingabe!
- Solche Probleme heißen
  - **pseudo-polynomial** bzw.
  - **schwach NP-vollständig (weakly NPC)**, falls sie NP-vollständig sind.

§4.6 Das Rucksackproblem

- Solche Probleme heißen
  - **pseudo-polynomial** bzw.
  - **schwach NP-vollständig (weakly NPC)**, falls sie NP-vollständig sind.



- Pseudo-polynomiale Probleme können für kleine Eingabegrößen relativ effizient gelöst werden.
  - Für große Eingabegrößen sind jedoch auch die Algorithmen zur Lösung pseudo-polynomialer Probleme für die Praxis zu ineffizient.